

LegalMind — Project Recap & Concept Reference

1. The Problem RAG Solves

LLMs only know what they were trained on. They've never seen your private contract, NDA, or company document. RAG lets you inject that document's content into the LLM's context **at query time**, so it can answer questions about information it was never trained on.

Without RAG: Question → LLM → generic/hallucinated answer

With RAG: Question → find relevant document chunks → LLM (question + chunks) → grounded answer

2. The Two Phases of RAG

Phase	When	What happens
Indexing	Once, when a document is uploaded	load → extract → chunk → embed → store in vector DB
Retrieval + Generation	Every time a user asks a question	embed question → search vector DB → send top-K chunks + question to LLM

3. Core Concept: Embeddings

An embedding is a list of numbers (a vector) that represents the **meaning** of a piece of text. Text with similar meaning produces vectors that are close together in that numerical space — even if they share no words in common.

Key facts learned:

- A whole chunk is embedded as **one** vector, not word by word
- `all-MiniLM-L6-v2` produces vectors of **384 dimensions**
- The number of dimensions is a tradeoff: more dimensions = richer meaning but slower/heavier; fewer = faster but less nuance
- The **same model** must be used to embed both the stored chunks and the incoming question — otherwise the two vectors live in incompatible "measurement systems" and can't be meaningfully compared (the cm vs inches analogy)

4. Chunking

Splitting a large document into smaller pieces before embedding.

Why chunking exists: precision, not speed. If a whole document is one vector, retrieval returns the entire document — too much noise for the LLM. Small chunks let retrieval return

just the 2-3 passages actually relevant to the question.

Key parameters:

- `chunk_size` — measured in **characters**, not words or sentences (~5 characters per word)
- `chunk_overlap` — characters shared between consecutive chunks so a sentence cut at a chunk boundary isn't stripped of context; conventionally **~10% of chunk_size**
- For legal text (long, complex sentences), larger chunks (~1000 characters) keep a full clause intact
- `RecursiveCharacterTextSplitter` tries natural breakpoints in priority order (paragraph → line → space → hard cut) before force-cutting at the character limit

5. Vector Database (ChromaDB)

Stores embeddings alongside the original text and metadata (like page number), and allows searching by **similarity** rather than exact match.

- `store()` — saves embeddings + original text + metadata
- `search()` — embeds the incoming question, then returns the **top-K** most similar stored chunks

K (top-K): how many chunks to retrieve.

- K too small → risk of missing relevant context
- K too large → noise dilutes the LLM's focus
- A practical starting point for legal Q&A: K=4

6. Why We Store Text *and* Vectors Together

The vector is only useful for **searching**. The original text is what gets shown to the user and sent to the LLM. Two different jobs, stored side by side in the same record.

7. Prompt Engineering for RAG

The prompt sent to the LLM combines the retrieved chunks (as context) with the user's question, plus behavioral rules:

- Stay grounded only in the provided document text
- Cite the page number for traceability
- Say "I don't know" rather than fabricate an answer if it's not in the document
- Ask for clarification if the question is ambiguous

This is what prevents hallucination — the LLM is explicitly restricted to the supplied context.

8. Full Pipeline Built

PDF → extract_text_from_pdf() → pages (with page numbers)

→ split_into_chunks() → chunks (with page + chunk numbers)

→ embed_chunks() → chunks + 384-dim vectors

→ store() → saved in ChromaDB

Question → search() → top-K relevant chunks from ChromaDB

→ build_prompt() → question + chunks + rules combined

→ generate_answer() → sent to Groq LLaMA → cited answer returned

9. Libraries & Tools Used

Library / Tool	Role in the project
<code>fitz</code> (PyMuPDF)	Extracts text from PDF, page by page
<code>langchain_text_splitters</code>	<code>RecursiveCharacterTextSplitter</code> — splits text into chunks intelligently
<code>sentence-transformers</code>	Loads <code>all-MiniLM-L6-v2</code> to turn text into 384-dim embeddings
<code>chromadb</code>	Vector database — stores embeddings, performs similarity search
<code>groq</code>	API client to call LLaMA 3.3 70B for answer generation
<code>python-dotenv</code>	Loads secrets (API keys) from <code>.env</code> without hardcoding them
<code>fastapi</code>	Exposes the pipeline as HTTP endpoints (<code>/upload</code> , <code>/ask</code> , <code>/health</code>)
<code>uvicorn</code>	ASGI server that actually runs the FastAPI app
<code>python-multipart</code>	Lets FastAPI parse uploaded files sent as <code>multipart/form-data</code>
<code>pydantic</code> (<code>BaseModel</code>)	Validates incoming request data has the right shape/type
<code>streamlit</code>	Builds the browser-based UI (upload button, question box, answer display)
<code>requests</code>	Lets Streamlit send HTTP requests to the FastAPI backend
Docker	Packages the app (code + dependencies + environment) so it runs identically anywhere
Docker Compose	Orchestrates the backend and frontend containers together as one system

10. New Engineering Concepts Learned

Virtual environments (`(venv)`) — isolate a project's Python packages from other projects, avoiding version conflicts.

`.env` vs `.env.example` — `.env` holds real secrets and is gitignored; `.env.example` is a template committed to the repo so collaborators know which variables to set.

Why load the embedding model once, outside the function — loading a model takes seconds; doing it inside a function that's called repeatedly wastes that cost every single call.

Why save uploaded files temporarily then delete them — `(fitz.open())` needs a real file path on disk, but an upload arrives as in-memory bytes; `save → process → delete` avoids accumulating unnecessary files.

Docker layer caching — Docker builds an image layer by layer and reuses unchanged layers on rebuild. Copying `(requirements.txt)` and running `(pip install)` *before* copying the rest of the code means code changes don't force a slow reinstall of every library.

`(127.0.0.1)` **inside a container** — refers to the container itself, not the host machine or other containers. Containers must reach each other using the **service name** defined in `(docker-compose.yml)` (e.g. `(http://backend:8000)`), which works because Compose puts them on a shared network.

`(depends_on)` vs `(healthcheck)` — `(depends_on)` alone only waits for a container to *start*, not for the service inside it to be *ready*. Adding a `(healthcheck)` (hitting `(/health)`) with `(condition: service_healthy)` makes the frontend wait until the backend can actually respond.

`(pip freeze)` **is dangerous for lightweight services** — it captures every installed package, including heavy unrelated ones (e.g. `(torch)`). A simple service like the Streamlit frontend should have its `(requirements.txt)` written manually with only what it truly needs.

11. What This Project Demonstrates (for portfolio/interviews)

- End-to-end RAG architecture (indexing + retrieval + generation)
- Embedding models and vector similarity search
- Prompt engineering to reduce hallucination and enforce citation
- REST API design with FastAPI
- Frontend/backend separation with a thin UI layer
- Containerization and multi-service orchestration with Docker Compose
- Debugging real infrastructure issues (caching, networking, health checks) — not just writing code that works once